Accelerating Qwen3-8B Decode on Tenstorrent Blackhole

A technical and statistical report on libtt's model-path optimizations

libtt performance study

11 July 2026

This report explains and measures the cumulative Qwen3-8B decode optimizations implemented in libtt's current TT-XLA/TT-MLIR/TTNN execution path. Nine revisions were evaluated with the serving command documented in `README.md`, greedy-sampling fusion disabled, two compile/warm-up requests discarded, and 32 independent 128-token requests retained per revision. Mean end-to-end generation throughput rises from 16.265 to 23.698 tokens/s, a 45.70% cumulative gain and a 31.36% reduction in mean request latency. The largest narrow change is runtime width-sharding of decode RMSNorm (+13.61%); a true Dst-resident matmul-SwiGLU epilogue adds +2.73%. The study also records two important negative results: generic two-way shared-LHS matmul fusion regresses by 1.69%, and fusing SiLU into a separate binary multiply has a -0.65% observed effect that is not significant after Holm correction. The final fallback-free simplification is performance-neutral.

Contents

1 Executive summary

The optimized branch `agent/qwen3-matmul-swiglu-epilogue-simplify` delivers **23.698 tokens/s** on the report workload, compared with **16.265 tokens/s** at the documented pre-optimization baseline. This is a **45.70% throughput increase**. Mean end-to-end latency for the fixed 128-token response falls from 7.870 s to 5.402 s, a **31.36% latency reduction**. An auxiliary streaming run on the final revision measured **25.790 pure decode tokens/s** (20 samples, 95% CI 25.657–25.923) after separating time to first token.

The principal findings are:

1. The initial decode-foundation bundle is decisive: it improves throughput by 22.58%. It combines compiler recognition of JAX RMSNorm and rank-3 RoPE, fixes KV-cache dtype propagation, enables BF8 activation lowering, and relaxes TTNN constraints needed by decode-specific layouts. Because these landed in one commit, the study attributes only their aggregate effect.
2. Recognizing JAX's *expanded* SiLU graph adds 3.30%, and fusing Qwen's decode QKV projection adds another 1.15%.
3. Broadening shared-LHS matmul fusion from three consumers to two is a statistically clear **-1.69% regression** in isolation. The combined gate/up representation is nevertheless an important enabler for the later SwiGLU epilogue. This is a useful distinction: an intermediate compiler form can be strategically valuable while being locally slower until its consumer is fused.
4. Runtime width-sharding of single-token RMSNorm over 16 cores is the largest single-purpose win at **+13.61%**.
5. Folding SiLU into a still-separate binary multiply is not enough. Its mean effect is -0.65%; the raw Welch test is nominally significant, but the result is not significant after correcting the eight adjacent comparisons (Holm-adjusted $p = 0.053$). The optimization avoids one intermediate, but it still writes and rereads the full gate/up matmul result.
6. The true matmul-SwiGLU epilogue is the successful fusion boundary. It keeps four gate and four up tiles together in the eight-tile Dst register set on each of 96 cores, applies SiLU and multiplication before packing, and writes only the half-width result. It contributes **+2.73%**.
7. Extending the matcher to the one-tile-row prefill case and removing the old fallback changes the mean by -0.19% ($p = 0.53$). It is therefore a code and coverage simplification with no detectable performance cost.

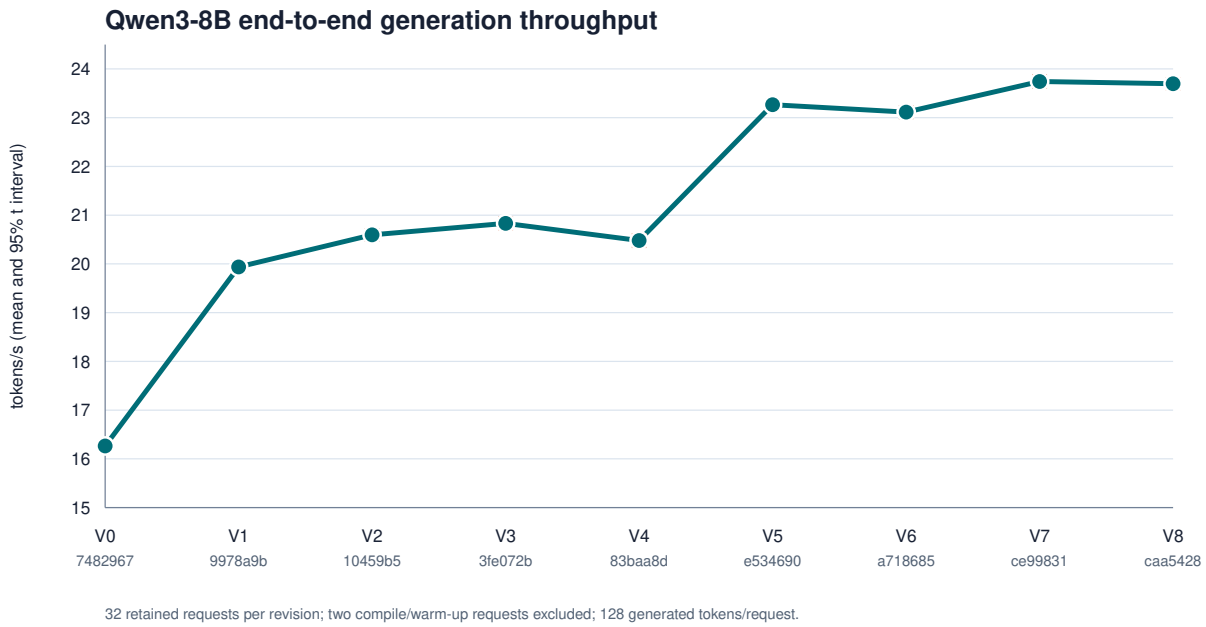

Figure 1.1: Cumulative throughput across the benchmarked revisions.

2 Scope and interpretation

2.1 What "each optimization" means in this report

The primary experiment covers every cumulative model-path optimization in the current, runnable Qwen3-8B serving line from commit `7482967` through commit `caa5428`. This is the sequence for which the same command in the current `README.md`, the same Qwen3-8B model, and the same TT-XLA/TTNN execution path remain meaningful. Each row is a real Git revision, not a feature flag applied to a shared binary.

libtt also has an older, locally implemented PJRT/compiler/runtime history. Those optimizations are inventoried in the historical section, but they are not assigned fabricated apples-to-oranges speedups. The local runtime was removed when libtt moved to upstream TT-XLA and TT-MLIR; older commits use different server entry points, model sizes, operation coverage, and runtime architecture. The present README benchmark cannot execute them without porting modern SGLang-JAX backward across that architectural boundary.

Sampling-path branches are intentionally excluded. The benchmark forces `SGLANG_TT_FUSED_GREEDY_SAMPLING` honoring the study's focus on the main model rather than token selection. Experimental branches that change the numeric contract, such as `agent/qwen3-mlp-bfp4`, require a separate quality and performance study rather than being mixed into the BF8-weight sequence.

2.2 Cumulative rather than factorial attribution

For variant i, the reported incremental speedup is

$$S_i = 100 \left(\frac{\bar{T}_i}{\bar{T}_{i-1}} - 1 \right),$$

where $\bar{T}_i$ is the arithmetic mean of the 32 per-request throughput measurements. Cumulative speedup replaces $\bar{T}_{i-1}$ with the V0 mean. This answers "what did this commit add to the stack immediately below it?" It does not claim that the effects commute: compiler and kernel fusions frequently interact, as V4 and V7 demonstrate.

3 Tenstorrent technical background

3.1 End-to-end software stack

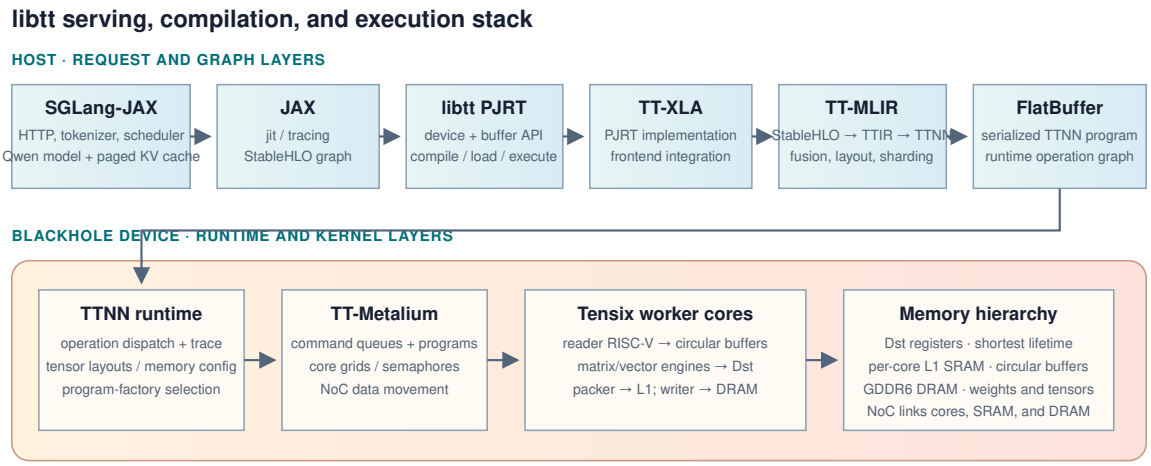

Figure 3.1: The software and hardware layers involved in a libtt generation request.

The request crosses the following layers:

1. **SGLang-JAX** accepts HTTP requests, tokenizes the prompt, schedules prefill/decode batches, owns the paged KV cache, and runs the JAX Qwen model. Its documented architecture separates the HTTP server, tokenizer manager, scheduler, model runner, and detokenizer manager.[1]
2. **JAX** traces the model computation and presents a StableHLO graph. StableHLO is a portable, ML-oriented operation set between frameworks and compilers.[2]
3. **libtt** is loaded as the `tt` PJRT plugin. PJRT deliberately makes device implementations opaque to frameworks: JAX calls a uniform compile, buffer, transfer, and execute interface while libtt/TT-XLA supplies the Tenstorrent-specific implementation.[3]
4. **TT-XLA and TT-MLIR** lower StableHLO through TTIR and TTNN dialects. TTIR carries tensor computation in a hardware-aware compiler IR; TTNN IR selects runtime operations, layouts, memory configurations, and device data types. The compiler serializes an executable FlatBuffer consumed by the TTNN runtime. Tenstorrent's official stack description identifies TT-XLA as the JAX/PyTorch PJRT frontend and TT-MLIR as the compiler that performs fusion, sharding, and layout lowering.[4]
5. **TTNN** is the high-level device-operation library. A TTNN matmul, RMSNorm, SDPA, or layout conversion selects a validated device operation and a program factory.

[1] SGLang-JAX project and architecture, including its core-structure documentation.
[2] OpenXLA, StableHLO specification.
[3] OpenXLA, PJRT — Uniform Device API.
[4] Tenstorrent, TT-Forge architecture and subprojects.

6. **TT-Metalium** creates command-queue programs containing per-core dataflow and compute kernels. It manages core ranges, circular buffers, semaphores, NoC transfers, and runtime arguments.
7. **Blackhole Tensix cores** execute those programs against device DRAM and per-core L1 SRAM.

This layering explains why libtt's changes appear as patches to TT-XLA, TT-MLIR, and TT-Metal rather than only to libtt's thin PJRT surface. A graph pattern has to be recognized in TTIR, represented in TTNN/FlatBuffer, selected by the TTNN runtime, and finally executed by a suitable Metal program.

3.2 Blackhole and the Tensix execution model

The test system contains a Blackhole P150. Current P150 firmware exposes 120 Tensix cores; Blackhole integrates GDDR6 memory, a two-dimensional NoC, and independent SRAM-rich worker cores.[5] The report's specialized SwiGLU program intentionally uses 96 workers, laid out within an 11-by-9-capable worker grid, rather than assuming every exposed core is available.

A Tensix core behaves more like a small dataflow computer than a conventional GPU streaming multiprocessor. A typical program has:

- a **reader data-movement kernel** that fetches tiles from DRAM or another core over the NoC into L1 circular buffers;
- a **compute kernel** that unpacks tiles into compute registers, invokes the matrix/vector engines, and produces result tiles;
- a **writer data-movement kernel** that drains result circular buffers to the destination tensor.

Circular buffers are bounded producer/consumer queues shared by the core's threads. The standard protocol reserves space at the back, pushes produced tiles, waits for tiles at the front, and pops consumed tiles.[6] Reader, compute, and writer kernels can therefore overlap on different data. Tenstorrent's matmul lab describes the same three-kernel pipeline and the NoC path from DRAM to fast core-local SRAM.[7]

3.2.1 Tiles, faces, and padding

TTNN's default tile is 32 by 32 elements. It is divided into four 16-by-16 faces because the matrix engine operates natively on 16-by-16 pieces. Tile layout pads the final two tensor dimensions to tile boundaries.[8] Consequently, batch-one decode still presents one full tile row to many operations: the logical sequence length is one, but the physical compute unit is a 32-row tile. Optimizations must reason about both logical shapes and padded tile shapes.

[5] Tenstorrent, Blackhole PCIe card documentation and P150 core-count firmware note.
[6] Tenstorrent, Circular Buffer APIs.
[7] Tenstorrent, Single Core Matrix Multiplication lab.
[8] Tenstorrent, TTNN Tensor: layout, tiles, and faces.

3.2.2 Dst registers and why epilogues matter

The Dst register set is the compute kernel's primary workspace. Matrix-engine results land in Dst; the vector engine can read and modify them; the packer moves them back to an L1 circular buffer. Conversely, the unpacker loads L1 tiles into Dst. The TT-Metal documentation explicitly identifies Dst as the matrix destination and vector source/destination, with `pack_tile` moving a result back to L1.[9]

Every boundary crossed after Dst costs work: pack, reserve/push a circular buffer, possibly write through the NoC to DRAM, launch another operation, read the tensor again, and unpack. A true epilogue wins by applying the consumer while producer values are already resident in Dst. Merely placing two TTNN operations in one graph region is not equivalent if the first operation's full output is still materialized.

3.2.3 DRAM, L1, interleaving, and sharding

Weights and large persistent tensors live in device DRAM. L1 is smaller and faster, and exists independently on each worker. A tensor's TTNN `MemoryConfig` specifies both buffer location and layout:

- **interleaved** storage distributes pages through memory banks;
- **height sharding** splits rows across core L1s;
- **width sharding** splits columns across core L1s;
- **block sharding** partitions both dimensions.

TTNN documents L1 sharding as distributing one shard to each core's L1, with a core grid, shard shape, strategy, and orientation defining the mapping.[10] `ttnn.to_memory_config` performs interleaved/sharded and DRAM/L1 conversions.[11] Sharding helps only when the parallel operation saves more than those conversion costs. V5 is a successful example because RMSNorm's reduction and affine work benefit enough from 16-way width sharding even after conversion in and out.

3.2.4 BF16 and BFP8 weights

The benchmark keeps activations and outputs in BF16 but asks TT-XLA to lower eligible model weights to `bfp_bf8` (`BFLOAT8_B`). BFLOAT8_B is a block-floating format: groups of 16 values share an exponent.[12] It reduces weight bytes and therefore decode's dominant DRAM traffic, at the cost of precision and a different reduction path. The SwiGLU kernel is deliberately specialized for BF16 input/output and BFLOAT8_B weights; it uses BF16 packer-L1 accumulation instead of FP32 Dst accumulation.

Packer-L1 accumulation stores partial sums in an L1 circular-buffer slot and adds later Dst values into that slot. TT-Metal exposes this explicitly as an L1-accumulation mode of the packer.[13] It permits a K dimension larger than the Dst capacity while reserving all eight Dst tiles for paired gate/up outputs.

[9] Tenstorrent, Compute engines and data flow within Tensix.
[10] Tenstorrent, TTNN Tensor sharding.
[11] Tenstorrent, `ttnn.to_memory_config`.
[12] Tenstorrent, TTNN BFLOAT8_B description and limitations.
[13] Tenstorrent, `pack_reconfig_l1_acc`.

3.3 Why autoregressive decode is special

Prefill processes many prompt tokens at once and exposes a large matmul M dimension. Decode processes one new token per active sequence, so at batch one most projections are logically $1 \times K$ by $K \times N$. The weights are revisited for every token, arithmetic intensity is low, and launch/data-motion overheads are a larger fraction of token time. The 32-row tile padding also makes naive kernels perform work on rows that are not logically populated.

For a simplified Qwen decoder layer,

$$\begin{aligned} u &= \mathrm{RMSNorm}(h), \\ h' &= h + \mathrm{Attention}(Q(u), K(u), V(u);\, K_{cache}, V_{cache}), \\ v &= \mathrm{RMSNorm}(h'), \\ g &= vW_{gate}, \qquad r = vW_{up}, \\ h_{next} &= h' + (\mathrm{SiLU}(g) \odot r)W_{down}. \end{aligned}$$

The repeatedly executed hot path therefore contains RMSNorm reductions, QKV projection/splitting, RoPE, paged-cache attention, paired gate/up matmuls, SiLU/multiply, a down projection, and residual additions. The optimizations in this report either recognize these graphs, choose decode-appropriate layouts, or eliminate materialization between their operations.

4 Optimization sequence

4.1 V0 — documented baseline (`7482967`)

V0 is the last documentation-only commit before the present decode series. It already includes the upstream TT-XLA/TT-MLIR architecture, Qwen3 SGLang-JAX support, BF8 weight lowering, fast tilize patches, and reduced cold-compile work. It is therefore a strong, runnable baseline—not the project's original unoptimized runtime.

4.2 V1 — decode foundation bundle (`9978a9b`)

This commit introduces several mutually dependent changes:

- a TTIR matcher collapses JAX's decomposed RMSNorm expression into a TTNN RMSNorm operation;
- RoPE fusion accepts the rank-3 decode shapes produced by JAX, reshaping to the rank-4 composite form and back;
- KV-cache dtype conversion propagates return types through cache-update ops;
- single-chip activation dtype lowering and TT-XLA compile options enable BF8 activation/weight lowering in the intended regions;
- TT-Metal layernorm validation accepts the single-core height-sharded decode case;
- SDPA decode accepts an L1-interleaved, non-sharded query tensor;
- SiLU lowering bridges the JAX call form used at this point in the sequence.

The aggregate gain is **+22.58%**. No sub-feature number is claimed because they are not separate commits and several are enabling correctness/layout conditions for the others.

4.3 V2 — expanded-SiLU recognition (`10459b5`)

JAX does not necessarily preserve SiLU as a named operation. The observed StableHLO/TTIR graph expresses it as

$$\mathrm{SiLU}(x) = x\,\sigma(x) = \frac{x}{1+\exp(-x)},$$

with typecasts, reshapes, broadcasts, and splatted constants around the scalar one. V2 looks through those view-like operations, proves the constant is one, matches divide/add/exp/neg, and replaces the outer multiply with a SiLU op. It removes a chain of elementwise launches and intermediates while retaining a strict same-input check for x and the sigmoid-like expression. The measured increment is **+3.30%**.

4.4 V3 — Qwen decode QKV projection fusion (`3fe072b`)

V3 recognizes the exact decode projection structure:

```
matmul → Q slice/reshape → Q RMSNorm → [1, B, Hq, D]
       → K slice/reshape → K RMSNorm → [1, B, Hkv, D]
       → V slice/reshape             → [1, B, Hkv, D]
```

The pattern verifies rank, contiguous Q/K/V bounds, head counts, head dimension, and operation roles. It then reshapes the projection once, invokes `nlp_create_qkv_heads_decode`, and recreates Q/K RMSNorm at rank four. This avoids materialized slices and reshapes in the decode loop. The measured gain is **+1.15%**.

4.5 V4 — two-way shared-LHS matmul fusion (`83baa8d`)

TTIR already fused groups of at least three matmuls that share a left-hand operand. V4 changes the validity threshold from three to two. That admits Qwen's paired gate/up projections, conceptually transforming

$$xW_{gate},\ xW_{up} \quad \longrightarrow \quad x\,[W_{gate}\ W_{up}].$$

The result is a **-1.69% regression** (Holm-adjusted $p = 8.81 \times 10^{-5}$). The likely mechanism is visible from the graph: V4 reduces launch/input-read duplication but still writes the doubled-width projection and then slices/reads it for SwiGLU. Wider output handling and the chosen matmul geometry cost more than the saved launch on this shape.

This result does not make V4 useless. V7 consumes precisely the combined gate/up representation and removes its expensive output boundary. V4 is an enabler whose isolated form is slower.

4.6 V5 — decode RMSNorm runtime sharding (`e534690`)

V5 detects a device-resident, tiled, unsharded RMSNorm input with logical sequence length one and width at least 2048. If the width divides evenly into 16 tile-aligned shards, it:

1. constructs an 8-by-2, row-major core grid;
2. width-shards the input into the 16 cores' L1 memories;
3. derives the layernorm program configuration from the shard specification;
4. runs TTNN RMSNorm in the sharded configuration; and
5. converts the output back to the requested/original memory configuration.

The measured increment is **+13.61%**, the largest narrow optimization in the study. At sequence length one, the generic interleaved path leaves too little parallel work in the height dimension. Width sharding exposes the hidden-size reduction across cores and more than repays two memory-config conversions.

4.7 V6 — SiLU fused into binary multiply (`a718685`)

The TTNN fusing pass detects a single-use SiLU feeding either side of a multiply. It removes the standalone SiLU op, marks the relevant BinaryNg input with a unary SiLU activation, extends the FlatBuffer enum, and calls TTNN `multiply` with a per-input activation list.

This is a legitimate elementwise fusion, but not a matmul epilogue. The full combined gate/up projection still reaches memory before BinaryNg consumes it. The observed effect is **-0.65%**. Its raw Welch p-value is 0.0266, but the Holm-adjusted p-value is 0.0533, so the experiment does not establish a family-wise-significant regression. It certainly provides no evidence of a speedup.

4.8 V7 — true Dst-resident matmul-SwiGLU epilogue (`ce99831`)

V7 adds a TT-MLIR pattern, TTNN `matmul_swiglu` operation, and dedicated TT-Metal program. Its contract is intentionally narrow:

- BF16 activation and output;
- BFLOAT8_B weight matrix;
- DRAM-interleaved tensors;
- no transpose, bias, or caller-provided output;
- one physical tile row;
- a grid supporting the selected 96 workers;
- half of the output tile width divisible across those workers.

For Qwen3-8B, each worker owns four gate-output tiles and four corresponding up-output tiles. Together they exactly fill the eight-tile Dst register set. The program:

1. uses two sender cores to multicast activation K blocks over the NoC;
2. has each of 96 workers read its paired gate/up BF8 weight tiles;
3. accumulates each K block into all eight Dst slots;
4. packs partial sums into L1 with BF16 packer-L1 accumulation;
5. reloads the final four gate and four up partials into Dst;
6. applies SiLU to Dst slots 0–3;
7. multiplies each by slots 4–7 in place; and
8. packs only four final BF16 tiles.

The critical difference from V6 is that the full gate/up matmul output never becomes a standalone tensor. The epilogue halves output traffic at the producer boundary and avoids a second operation's read/unpack path. The measured gain is **+2.73%**.

4.9 V8 — prefill-capable, fallback-free simplification (`caa5428`)

V8 generalizes the compiler matcher from decode-only naming/shape assumptions to any supported one-tile-row input, covering short prefill as well as decode. It removes the V6 SiLU-multiply patch and its fallback path, leaving the true matmul epilogue as the single implementation for the matched form. Validation remains in TTNN/TT-Metal, where unsupported dtype/layout/shape combinations are rejected rather than silently taking a slower path.

The measured difference is **-0.19%**, p = 0.532. The result is statistically neutral and its confidence interval overlaps V7's closely. V8 therefore achieves the intended simplification and broader match coverage without a detectable throughput regression.

5 Benchmark methodology

5.1 System and software

Item	Value
Accelerator	Tenstorrent Blackhole P150
Firmware observed at startup	19.6.0
Host CPU	Intel Core Ultra 9 185H, 22 logical CPUs online
Host OS/kernel	Linux 6.8.0-124-generic, x86-64
Model	`Qwen/Qwen3-8B`
Model dtype	BF16
Weight-lowering request	`bfp_bf8`
Attention backend	TT
Serving frontend	`/home/pcmoritz/sglang-jax`
PJRT plugin	revision-specific `bazel-bin/libtt.so`
Final report revision	`caa5428fb86c9ffc6a9dda8685126c5ac4353013`
Benchmark date	11 July 2026 (America/Los_Angeles)

V0–V4 were rebuilt and collected in one sequential benchmark session. V5–V8 use the first 32 retained observations from the existing 40-observation runs collected earlier on the same date, host, accelerator, model, server command, and sampling-disabled configuration. All raw observations used in the report are normalized into `data/samples.csv`.

5.2 Server configuration

Each revision was checked out, built with its revision-appropriate Bazel target (`//:tt`, producing `bazel-bin/libtt.so`), and served with the README command. The effective command was:

```
env -u TT_METAL_RUNTIME_ROOT \
  PYTHONPATH=/home/pcmoritz/sglang-jax/python \
  PJRT_NAMES_AND_LIBRARY_PATHS=tt:/home/pcmoritz/libtt/bazel-bin/libtt.so \
  JAX_PLATFORMS=tt \
  JAX_USE_SHARDY_PARTITIONER=false \
  JAX_COMPILATION_CACHE_DIR=/tmp/sglang-jax-qwen3-8b-jax-cache \
  SGLANG_TT_HOST_WEIGHT_LOAD=1 \
  SGLANG_TT_OPTIMIZATION_LEVEL=1 \
  SGLANG_TT_EXPERIMENTAL_WEIGHT_DTYPE=bfp_bf8 \
  SGLANG_TT_FUSED_GREEDY_SAMPLING=0 \
  /home/pcmoritz/sglang-jax/.venv/bin/python -m sgl_jax.launch_server \
  --model-path Qwen/Qwen3-8B \
  --host 127.0.0.1 --port 31000 \
```

```
  --device tt --dtype bfloat16 --attention-backend tt \
  --max-running-requests 2 --max-total-tokens 1024 \
  --max-prefill-tokens 256 --chunked-prefill-size 256 \
  --page-size 32 --watchdog-timeout 1200 \
  --disable-precompile --skip-server-warmup \
  --disable-overlap-schedule --disable-radix-cache
```

Disabling radix cache ensures every request executes the 32-token prompt prefill rather than reusing a prefix. Disabling overlap schedule removes cross-request scheduler overlap. The maximum of two running requests is a server capacity setting; requests were issued serially. Fused greedy sampling is explicitly disabled so improvements reflect the model path.

5.3 Request and sample policy

The identical request was issued 34 times per revision:

```
curl -fsS http://127.0.0.1:31000/generate \
  -H 'Content-Type: application/json' \
  -d '{
    "text": "The capital of France is",
    "sampling_params": {"temperature": 0, "max_new_tokens": 128}
  }'
```

The first two responses were discarded. They contain graph compilation, TT-Metal kernel compilation, trace setup, and cache population; they are not steady-state samples. The next 32 complete responses form the analysis window. Every retained request returned exactly 128 token IDs.

For request (j), throughput is

$$T_j = \frac{128}{L_j},$$

where (L_j) is SGLang's `meta_info.e2e_latency`. This includes request-local prefill, decode, and serving overhead, so it is intentionally an end-to-end generation metric rather than a kernel-only number.

5.4 Statistical treatment

For each revision the report gives the arithmetic mean, sample standard deviation, median, range, and a two-sided 95% Student-t confidence interval for the mean. Adjacent revisions are compared with a two-sided Welch t-test, which does not assume equal variance. Eight planned adjacent comparisons create a multiple-testing family, so Holm's step-down method controls family-wise error. Raw and adjusted p-values are preserved in `data/summary.csv`.

The tests assume independent observations. Sequential accelerator timings can exhibit autocorrelation and thermal drift, so p-values should be read as descriptive evidence for this run, not as a universal hardware guarantee. The effect sizes and confidence intervals are more important than crossing a single threshold.

6 Results

6.1 Primary throughput results

Table 6.1: End-to-end 128-token generation results, 32 retained requests per revision. "Holm p" is the multiplicity-adjusted p-value for the comparison with the preceding row.

Variant	Commit	Mean ± SD (tok/s)	95% CI	Incremental	vs. V0	Holm p
V0	`7482967`	16.265 ± 0.128	[16.218, 16.311]	—	0.00%	—
V1	`9978a9b`	19.938 ± 0.283	[19.836, 20.040]	**+22.58%**	+22.58%	2.33e-44
V2	`10459b5`	20.596 ± 0.279	[20.495, 20.696]	**+3.30%**	+26.63%	1.09e-12
V3	`3fe072b`	20.832 ± 0.293	[20.726, 20.938]	**+1.15%**	+28.08%	0.00481
V4	`83baa8d`	20.479 ± 0.321	[20.363, 20.594]	**-1.69%**	+25.91%	8.81e-5
V5	`e534690`	23.265 ± 0.235	[23.181, 23.350]	**+13.61%**	+43.04%	2.85e-42
V6	`a718685`	23.113 ± 0.296	[23.007, 23.220]	-0.65%	+42.11%	0.0533
V7	`ce99831`	23.744 ± 0.311	[23.632, 23.856]	**+2.73%**	+45.98%	5.90e-11
V8	`caa5428`	23.698 ± 0.274	[23.599, 23.797]	-0.19%	**+45.70%**	0.532

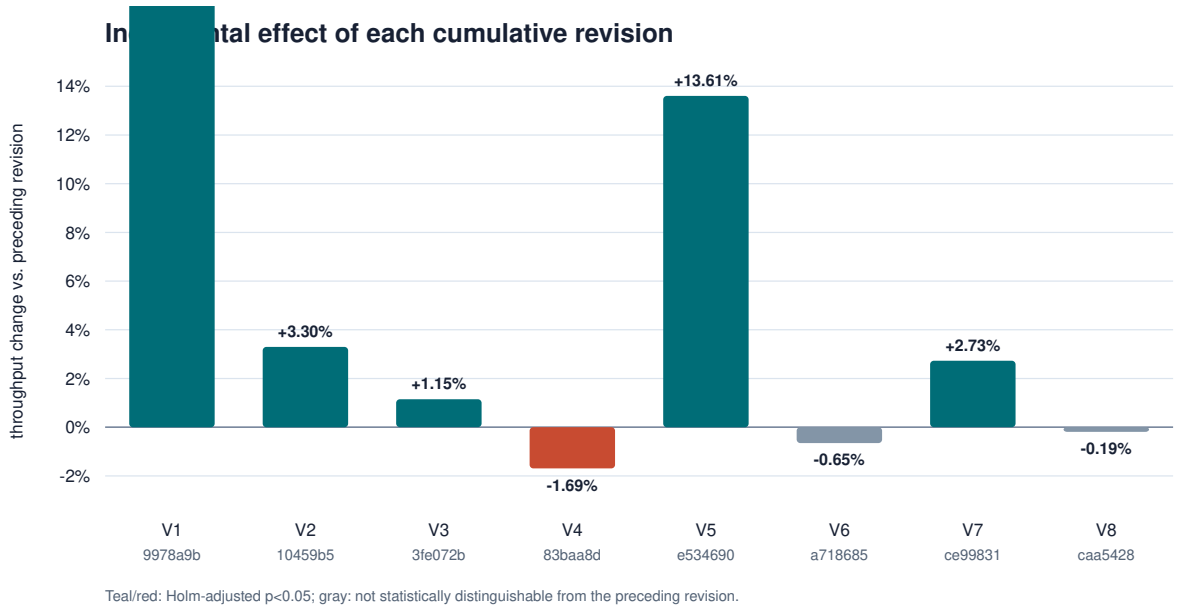

Figure 6.1: Incremental speedup or regression introduced by each revision.

The final mean is slightly below V7, but the difference is only 0.046 tok/s and is not statistically distinguishable from zero. In contrast, V4's confidence interval sits below V3's and its adjusted p-value is small. The data therefore support "V8 preserves V7 performance," but not "every fusion improved performance."

6.2 Pure decode cross-check

The `/generate` response reports one end-to-end latency. A separate streaming run on V8 recorded the first token and remaining token intervals. After two warm-ups, 20 retained observations yielded:

Metric	Mean ± SD	95% CI
Time to first token	0.6105 ± 0.0043 s	[0.6085, 0.6125] s
Total streaming time	5.5355 ± 0.0544 s	[5.5100, 5.5610] s
Pure decode throughput	25.790 ± 0.284 tok/s	[25.657, 25.923] tok/s

The 25.79 tok/s number is higher than the 23.70 end-to-end metric because it excludes prompt processing and first-token latency. It is a useful description of the final decoder but is not used for inter-revision attribution because the full streaming experiment was not repeated for every revision.

6.3 Correctness and numeric stability checks

Within every variant, all 32 retained requests produced the same 128 token IDs. The SHA-256 prefixes of those ID sequences were:

Variants	Token-ID hash prefix
V0–V1	`68e10a82edac`
V2–V3	`1ab55f92178d`
V4–V6	`808d3949246a`
V7–V8	`2eb88f74c7d9`

The transitions align with compiler/numeric changes and do not imply request nondeterminism. Greedy decoding is sensitive to tiny logit reorderings: BF8 lowering, operation reassociation, reduction order, or a new kernel can change a later argmax. This performance report verifies shape/completion and within-variant determinism; it is not a model-quality evaluation. A production decision should add perplexity and task-level quality suites.

7 Reading the results mechanistically

7.1 Launch reduction is not the same as traffic reduction

V4, V6, and V7 form the clearest controlled lesson in the study:

```
V4: combine two matmuls
    → fewer launches/input reads
    → still materialize doubled gate/up output
    → -1.69%

V6: combine SiLU with the following multiply
    → remove standalone SiLU output
    → still materialize and reread matmul output
    → -0.65% observed; adjusted result inconclusive

V7: make SwiGLU the matmul epilogue
    → gate/up stay in Dst
    → write only final half-width tensor
    → +2.73%
```

At batch-one decode, model weights and intermediate tiles dominate movement. Eliminating an operation launch is helpful only if the fused program does not create a worse geometry or preserve the same expensive tensor boundary. The successful fusion boundary is producer-side, before packing the matmul result.

7.2 Why RMSNorm sharding is unusually large

RMSNorm performs square, reduction over hidden width, reciprocal square root, and affine scaling. With logical height one, a generic height-parallel scheme cannot occupy many cores. V5's width sharding creates 16 independent L1 shards and selects a sharded layernorm program. The hidden width is large and repeats in every decoder layer, so this local change compounds across the model.

The implementation is also conservative: it activates only for a device, tiled, unsharded tensor; sequence length one; width at least 2048; and a width divisible into 16 tile-aligned shards. Prefill and already-sharded cases retain their prior behavior.

7.3 Why the final simplification is safe for performance

V8 does not rewrite the TT-Metal compute loop that produced V7's speedup. It changes compiler matching/coverage and deletes the V6 fallback. The retained V7 and V8 output hashes are identical, their mean difference is small, and the adjusted p-value is 0.532. Those three signals support the claim that the code simplification preserves both the observed token sequence and runtime performance for this workload.

8 Historical optimization inventory

Before the upstream TT-XLA/TT-MLIR transition, libtt implemented substantially more of the compiler and runtime locally. The following first-parent commits are performance-relevant milestones. They remain important engineering context even though the modern README benchmark cannot isolate them.

Commit	Historical optimization	Technical intent
`701dc4e`	Fast dispatch	Replace slow command submission with the device fast-dispatch path.
`f4df790`	Optimized matmul	Introduce the first performance-oriented tiled matmul implementation.
`0f39cae`	Parallel binary elementwise	Divide elementwise tiles across cores.
`6f052ba`	Qwen3 model optimization	Add model-specific graph/runtime improvements in the local stack.
`8678414`	DRAM/broadcast fast paths	Specialize common broadcast/data-movement forms.
`1d80118`	Deferred finish and batched CQ writes	Avoid host synchronization after each dispatch and batch command-queue writes.
`1875113`	Fused elementwise lowering	Compile elementwise expression trees as fewer device programs.
`2778027`	Matmul top-1 epilogue	Perform top-1 selection at the logits matmul producer boundary.
`9fc9a36`	Cached decode attention	Specialize attention for autoregressive KV-cache reuse.
`29f685a`	Allocator locking split	Reduce lock scope around runtime readback/allocation work.
`d1b422e`	Reshape optimization	Turn eligible reshapes into views or cheaper movement.
`65cd673`	Transpose into matmul	Fold operand transpose into matmul reads.

Commit	Historical optimization	Technical intent
`f05995a`	Fused SDPA decode	Replace decomposed attention with a decode SDPA device operation.
`f7ba52b`	Lazy reshape propagation	Carry views through gather/scatter instead of materializing.
`a23a624`	Fused RMSNorm	Replace decomposed normalization with a dedicated kernel.
`eee55b7`	Tile-transpose matmul reads	Improve source tile lookup and NoC reads for transposed operands.
`119aab5`	Fused RoPE	Collapse rotary embedding's elementwise/reshape graph.
`f2e705f`	SDPA over interleaved KV cache	Avoid cache layout conversion before decode attention.
`08423ec`	Parallel BF16 top-1	Spread logits argmax/top-1 work across cores.
`94734cc`	TT-MLIR tilize patches	Reduce layout-conversion overhead after the upstream-stack transition.
`eb8c90d`	Cold-compile optimization	Remove/avoid expensive host/compiler work on the first execution.

Several exploratory branches also record useful negative or orthogonal work: residual-as-bias experiments, slice-free SwiGLU, sampling fusion, logits untilize/top-1 fusion, DRAM-sharded matmul, and BFP4 MLP weights. Only changes that survive in the V0–V8 ancestry are assigned numbers in the primary table. This prevents an abandoned prototype, a sampling optimization, or a changed precision mode from being represented as part of the final model-path speedup.

9 Limitations and follow-up work

1. **One model and shape.** Results apply to Qwen3-8B, serial batch-one-style generation, a 32-token prompt, and 128 generated tokens. Larger batch, longer context, or multi-user continuous batching can move the bottleneck.
2. **End-to-end metric.** The primary metric contains short prefill and server overhead. The streaming cross-check isolates final decode, but only on V8.
3. **Sequential ordering.** Revisions were not randomized or interleaved. Thermal state and background load can create drift. A publication-grade follow-up would use randomized blocks with repeated server restarts.
4. **Autocorrelation.** Welch/t intervals treat requests as independent. A blocked bootstrap over time or heteroskedasticity/autocorrelation-consistent model would be more conservative if long runs show serial correlation.
5. **One accelerator.** The specialized epilogue assumes Blackhole's available grid and the Qwen3-8B tile geometry. Wormhole, multi-chip, or another MLP width requires its own program configuration.
6. **Quality is not measured.** Token determinism is checked within variants, but cross-variant hashes differ. Perplexity and downstream evaluations are necessary before changing precision or reduction order in production.
7. **The V1 bundle is not decomposed.** Its +22.58% cannot be divided among RMSNorm, RoPE, KV dtype, SDPA layout, and BF8-enabling changes without constructing additional cherry-picked revisions.

The most promising next optimization is to keep more of the MLP pipeline on chip: consume the SwiGLU output directly in a compatible down-projection path, or retain a sharded layout between them. That requires coordinating output shards and down-projection weight distribution; simply attaching more unary work to BinaryNg is unlikely to reproduce V7's producer-side benefit.

10 Reproduction and report generation

The report bundle contains:

- `report.md`: this source document;
- `data/samples.csv`: all 288 retained per-request observations;
- `data/summary.csv`: descriptive statistics, speedups, raw Welch p-values, Holm-adjusted p-values, and output hashes;
- `analyze.py`: the exact analysis and SVG generation code;
- `figures/*.svg`: resolution-independent figures;
- `style.css` and `Makefile`: HTML, LaTeX, and PDF build paths.

To recompute statistics from the raw `/tmp` JSON directories used on the benchmark host:

```
/home/pcmoritz/sglang-jax/.venv/bin/python \
  docs/libtt-optimization-report/analyze.py
```

To build a self-contained HTML report:

```
make -C docs/libtt-optimization-report html
```

For a typeset PDF, install a XeLaTeX distribution and `rsvg-convert` (commonly packaged as `librsvg2-bin`), then run:

```
make -C docs/libtt-optimization-report pdf
```

The `pdf-html` target is an alternative for environments with WeasyPrint. SVG figures, vector text, booktabs-style tables, controlled page geometry, microtypography, numbered headings, and a generated table of contents are preserved by the PDF route.

11 Conclusion

The current libtt optimization line improves the documented Qwen3-8B serving workload by **45.70%**, from 16.265 to 23.698 tokens/s. The data favor three principles:

- recognize framework-generated algebra before lowering;
- choose layouts that expose decode's width parallelism; and
- place fusion at the producer epilogue so intermediate values never cross an avoidable memory boundary.

The final matmul-SwiGLU implementation is faster because it follows the third principle literally: gate and up values remain in Dst until `silu(gate) * up` is complete. The fallback-free simplification retains that performance. Just as importantly, the report preserves the regressions and neutral results, which prevents operation-count reduction from being mistaken for device-level speedup.